

Income Classification Project Report

1. Data Preprocessing

In this project, several preprocessing steps were applied to prepare the dataset for machine learning models.

The goal was to clean the data, handle outliers, encode categorical variables, and scale numerical features to ensure consistent and reliable model performance.

clean(df)

This function is used to clean the dataset by removing extra spaces and punctuation from string values. It also strips column names to avoid mismatches.

This step was necessary because there was an issue during transformation where some values in the test set were not recognized due to extra spaces or trailing dots in categorical values. This function ensures consistency between training and testing data so that encoding works correctly.

detect_outliers(column, df)

This function is used to detect and handle outliers in numerical columns using the IQR (Interquartile Range) method.

It calculates Q1 and Q3, then computes the IQR.

Based on these values, lower and upper bounds are defined. Any value below the lower bound is replaced with the lower bound,

and any value above the upper bound is replaced with the upper bound.

This approach is used instead of removing rows to preserve dataset size while reducing the effect of extreme values on the model.

labeldata(column, df)

This function applies Label Encoding during training.

It converts categorical values into numerical values. It is used only for binary columns such as `sex` and `Income`, because they contain only two unique values. Label Encoding is suitable here since it does not introduce misleading relationships in binary features.

labeldatatest(column, df)

This function applies Label Encoding on test data using the encoders learned from the training data. It does not fit again; it only transforms the values using the already stored encoders.

This ensures that the test data is encoded in the same way as the training data.

onehot_encoding(df, column)

This function applies One-Hot Encoding on training data. It converts categorical variables into multiple binary columns.

It is used for multi-class categorical features such as `workclass`, `education`, `occupation`, `relationship`, `race`, and `native-country`. One-Hot Encoding is used instead of Label Encoding to avoid introducing false ordinal relationships between categories.

onehot_encoding_test(df, column)

This function applies One-Hot Encoding on test data using the already fitted encoder from training. It only transforms the data without fitting again to ensure consistency between training and testing datasets.

scale (MinMaxScaler)

MinMaxScaler is used to normalize numerical features into a range between 0 and 1.

This is important because features such as `fnlwgt` and `capital-gain` have very large value ranges compared to other features.

Scaling ensures that all features contribute equally to the model. During training, `fit_transform` is used, while during testing only `transform` is applied to avoid data leakage.

encoders dictionary

This dictionary stores all LabelEncoders used during training. It is saved using joblib and later loaded during prediction to ensure that the same encoding logic is applied to new input data. This guarantees consistency between training and inference.

Saved Preprocessing Objects

At the end of preprocessing, the following objects are saved using joblib:

- `encoders.pkl` → stores LabelEncoders
- `onehotencoder.pkl` → stores OneHotEncoder
- `scale.pkl` → stores MinMaxScaler

These saved objects are necessary to apply the same transformations during model prediction without retraining the preprocessing pipeline.

2. Logistic Regression

Modeling and Hyperparameter Tuning

I selected Logistic Regression as the primary classifier for this income prediction task. To optimize the model's performance and understand its behavior under different constraints, I experimented with the 'C' parameter (the inverse of regularization strength).

Experimentation Results: Performance Metrics

Hyperparameter	Class	Precision	Recall	F1-Score	Accuracy
C = 0.01 (Strong Reg)	0	0.86	0.94	0.90	83.84%
	1	0.72	0.52	0.60	
C = 1.0 (Default)	0	0.87	0.92	0.90	83.84%
	1	0.70	0.56	0.62	

Hyperparameter	Class	Precision	Recall	F1-Score	Accuracy
C = 100 (Weak Reg)	0	0.87	0.92	0.90	83.82%
	1	0.69	0.56	0.62	

Confusion Matrix Analysis (Default Hyperparameter C = 1.0)

Actual Amount	Estimated 1 (Yes)	Estimated 0 (No)	Total
Actual 1 (Yes)	2,165 (TP)	1,681 (FN)	3,846
Actual 0 (No)	950 (FP)	11,480 (TN)	12,430
Total	3,115	13,161	16,276

Detailed Analysis of Hyperparameters

- Strong Regularization (C = 0.01):** By imposing a stricter penalty on coefficients, the model became more conservative. This led to the highest Recall for Class 0 (94%), ensuring high accuracy for low-income identification.
- Default Setting (C = 1.0):** This provided a robust baseline. As shown in the matrix above, the model successfully identified 2,165 high-income cases while maintaining balanced precision and recall.
- Weak Regularization (C = 100):** Reducing the regularization allowed the model to fit the data more aggressively, leading to a slight increase in True Positives (2,169) but also increasing False Positives (957).

Data Visualization & Analysis

Based on the feature coefficients from the Logistic Regression model, key predictors were identified:

- Education-num:** Strong positive correlation; higher education significantly increases the probability of earning >50K.
- Age:** Positive trend reflecting professional experience and career progression.
- Capital-gain:** Critical numeric factor for distinguishing individuals in the high-income bracket.

Conclusion

The Logistic Regression model performed consistently with an accuracy of approximately 84%. The evaluation shows that using the **Default Hyperparameter (C = 1.0)** ensures a balanced trade-off between precision and recall, making it an effective choice for identifying high-income individuals within this dataset.

3. Support Vector Machine (SVM)

Model Overview

For this classification task, I selected the **Support Vector Machine (SVM)**. This choice was driven by SVM's proven effectiveness in high-dimensional spaces and its ability to define complex, non-linear decision boundaries using kernel functions. Specifically, the **Radial Basis Function (RBF)** kernel was used to capture intricate patterns within the census data.

To optimize the model, I focused on the **'C' hyperparameter**, which acts as a regularization parameter. It controls the trade-off between achieving a low error on the training data and maximizing the margin of the decision boundary.

Evaluation Results (C = 1)

Using the default regularization strength ($C=1$), the model achieved a robust balance between precision and recall. The results on the test set are as follows:

Metric	Value
Accuracy	83.74%
Precision	70.15%
Recall	54.26%
F1-Score	61.19%

Confusion Matrix Analysis

	Predicted: <=50K	Predicted: >50K
Actual: <=50K	11,542 (TN)	888 (FP)
Actual: >50K	1,759 (FN)	2,087 (TP)

- **True Negatives (11,542):** The model accurately identifies the majority of individuals in the lower-income bracket.
- **False Positives (888):** A relatively small number of individuals were incorrectly predicted to earn >50K.
- **False Negatives (1,759):** Approximately 45% of high-income individuals were misclassified, reflecting the model's conservative nature in the face of class imbalance.
- **True Positives (2,087):** The model correctly identified over half of the high-income population.

Hyperparameter Analysis

To understand the effect of the regularization parameter **C**, experiments were conducted with three values: **C = 0.01**, **C = 1**, and **C = 100**.

Evaluation Metrics Across C Values

Hyperparameter (C)	Accuracy	Precision	Recall	F1-Score
C = 0.01 (Strong Reg)	82.56%	73.72%	40.69%	52.44%
C = 1 (Default)	83.74%	70.15%	54.26%	61.19%
C = 100 (Weak Reg)	82.52%	64.65%	57.44%	60.83%

Confusion Matrices Across C Values

C = 0.01

	Predicted: <=50K	Predicted: >50K
Actual: <=50K	11,872 (TN)	558 (FP)
Actual: >50K	2,281 (FN)	1,565 (TP)

C = 1

	Predicted: <=50K	Predicted: >50K
Actual: <=50K	11,542 (TN)	888 (FP)
Actual: >50K	1,759 (FN)	2,087 (TP)

C = 100

	Predicted: <=50K	Predicted: >50K
--	------------------	-----------------

	Predicted: <=50K	Predicted: >50K
Actual: <=50K	11,222 (TN)	1,208 (FP)
Actual: >50K	1,637 (FN)	2,209 (TP)

Observations

- **Low C (0.01):** Produced a "stiffer" boundary. While this increased precision (fewer false alarms), it severely hampered recall, as the model became too cautious about predicting the positive class (>50K).
- **High C (100):** Made the model more aggressive. While this improved recall (finding more high-income individuals), it led to a higher rate of false positives, effectively reducing overall precision.
- **Optimal C (1):** As shown in the results above, **C=1** provided the most balanced performance across all evaluation metrics, particularly for the F1-score.

Data Interpretation & Insights

- **Non-Linearity:** The effectiveness of the RBF kernel suggests that income levels are not separated by simple linear thresholds.
- **Class Imbalance:** The higher number of False Negatives compared to False Positives shows that the model is naturally biased towards the majority class.

Conclusion

The Support Vector Machine model reached an accuracy of **83.74%**. Through the analysis of the confusion matrix and hyperparameter tuning, it was determined that **C = 1** with an RBF kernel serves as the optimal configuration for this specific problem.

The final model has been serialized and saved as `SVMmodel.pkl` using the `pickle` library.

4. Decision Tree

Model Overview

A Decision Tree Classifier was used for the prediction task due to its interpretability and ability to handle both numerical and categorical data.

Manual Hyperparameter Tuning

Initially, different hyperparameters were tested manually:

- max_depth: [5, 6, 10]
- min_samples_leaf: [10]
- min_samples_split: [5, 10]
- class_weight: None vs balanced

Best Manual Model

- max_depth = 10
- min_samples_leaf = 10
- min_samples_split = 10

Performance

- TRAIN ACCURACY: 0.8465132003565172
- TEST ACCURACY: 0.8329442123371836

Class	Precision	Recall	F1-score	Support
0	0.87	0.91	0.89	12430
1	0.67	0.57	0.62	3846
Accuracy	-	-	0.83	16276
Macro Avg	0.77	0.74	0.76	16276
Weighted Avg	0.83	0.83	0.83	16276

Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	11352	1078
Actual 1	1641	2205

This model showed a good balance between accuracy and generalization, with limited overfitting.

Grid Search Optimization

After manual tuning, GridSearchCV was applied to automatically search for the best hyperparameter combination using cross-validation.

Parameter Grid

- max_depth: [6, 8, 10, 12]
- min_samples_leaf: [5, 10, 15]
- min_samples_split: [5, 10, 15]
- criterion: [gini, entropy]

Best Parameters Found

- max_depth = 8
- min_samples_leaf = 15
- min_samples_split = 5
- criterion = entropy

Performance

- TRAIN ACCURACY: 0.8504779174478286
- TEST ACCURACY: 0.8346645367412141

Class	Precision	Recall	F1-score	Support
0	0.88	0.91	0.89	12430
1	0.67	0.59	0.63	3846
Accuracy	-	-	0.83	16276
Macro Avg	0.77	0.75	0.76	16276
Weighted Avg	0.83	0.83	0.83	16276

Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	11314	1116
Actual 1	1575	2271

Better generalization compared to manual tuning.

Conclusion

In the Decision Tree model, manual hyperparameter tuning provided a strong baseline model with good accuracy and balanced performance. However, GridSearchCV further improved the model by systematically searching for the best combination of hyperparameters using cross-validation.

Although the improvement in accuracy was small, GridSearch provided a more reliable and optimized model configuration.